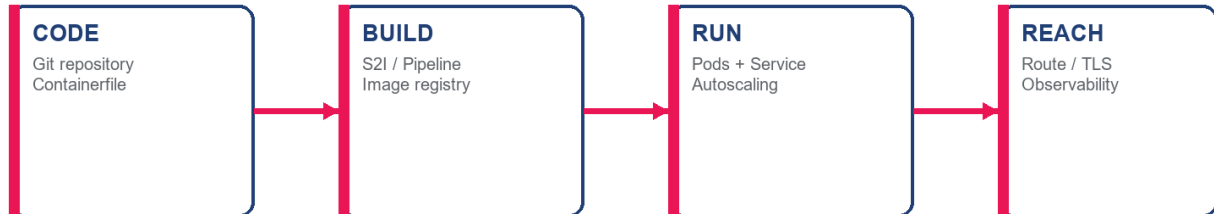


WORKSHOP HANDBOOK

OpenShift Essentials

Build, Deploy & Operate - a practical participant guide

A scenario-led guide to understanding the OpenShift platform, navigating projects, and deploying applications with confidence.



Workshop outcome	What you will be able to do
Explain	Describe how OpenShift extends Kubernetes into an enterprise application platform.
Navigate	Use projects, the web console, and oc to find and inspect resources.
Deploy	Follow a source-to-build-to-rollout workflow and expose an application safely.
Diagnose	Use events, logs, status, and resource relationships to locate common failures.

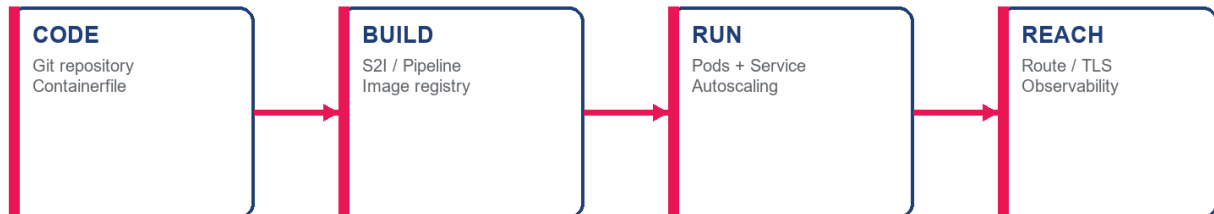
HOW TO USE THIS GUIDE Read the concept, trace the diagram, then work through the scenario and explain the result to a partner.

TOPIC 1

Introduction to OpenShift and Its Role in Modern IT

From container orchestration to a governed developer platform

OpenShift Container Platform is a Kubernetes-based platform that combines orchestration with integrated build, networking, security, observability, lifecycle, and developer workflows. Its value is a repeatable path from source code to a secure, observable service.



Modern IT pressure	OpenShift response	Practical result
Release faster	Automated builds and declarative rollouts	Smaller, repeatable releases
Run anywhere	Consistent Kubernetes API across supported infrastructure	Portable operating model
Control risk	RBAC, admission, SCCs, policies	Guardrails without ticket-driven deployment
Operate reliably	Health probes, metrics, logs, Operators	Faster detection and recovery

REAL-TIME SCENARIO A bank must release a payment API weekly while enforcing approved images, namespace quotas, TLS, and auditability. OpenShift provides one governed delivery path rather than a separate toolchain for every team.

Audience prompt: Which delivery step in your organization is currently manual, inconsistent, or dependent on a specialist?

TOPIC 1 - OPERATING MODEL

Why Enterprises Adopt a Platform

OpenShift as a product, not merely a cluster

A successful OpenShift adoption creates a paved road: supported ways to build, deploy, expose, observe, and update applications. The platform team owns reusable capabilities and guardrails; application teams retain ownership of code, configuration, reliability targets, and business behavior.

Platform capability	Platform team provides	Application team consumes
Identity and access	OAuth integration, groups, role patterns	Project-scoped access through approved roles
Software supply chain	Trusted builders, registries, signing and scanning	Reproducible builds from reviewed source
Runtime guardrails	SCCs, quotas, limits, network policy patterns	Explicit resource requests, probes and dependencies
Reliability	Ingress, monitoring, logging, update lifecycle	Service objectives, dashboards and runbooks
Self-service	Catalogs, templates, Operators and GitOps	Repeatable environments without infrastructure tickets

EXPERT VIEW Measure the platform by reduced cognitive load and safer lead time, not by cluster utilization alone. A cluster can be healthy while the developer experience remains fragmented.

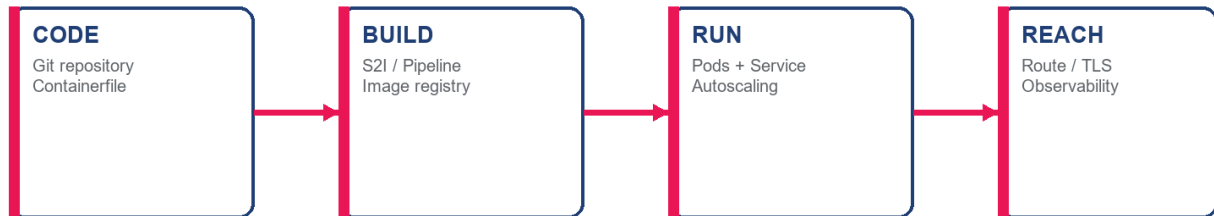
Anti-patterns to discuss: granting cluster-admin to unblock teams; treating every namespace as unique; building pipelines without image provenance; ignoring resource requests; and using the console without understanding the resource graph.

Workshop question: Which capabilities should be mandatory guardrails, and which should be optional accelerators?

TOPIC 1 - DELIVERY LIFECYCLE

The OpenShift Application Lifecycle

From commit to feedback and controlled recovery



Lifecycle stage	Primary objects/evidence	Expert questions
Source and build	Git commit, BuildConfig/PipelineRun, build logs	Is the source immutable? Which trigger started the build?
Image management	ImageStream, digest, registry metadata	Can the deployed image be traced to source and scan result?
Release	Deployment or DeploymentConfig, ReplicaSet/RC	Which strategy is used? What condition blocks progress?
Service exposure	Service, EndpointSlice, Route/Ingress	Do selectors and ports resolve to ready endpoints?
Runtime assurance	Probes, requests/limits, metrics, logs, alerts	What does healthy mean to users? Is rollback evidence available?

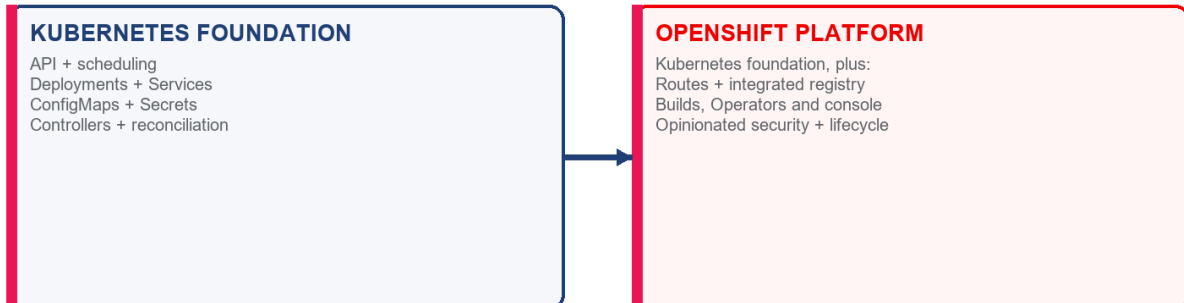
REAL-TIME SCENARIO A new image is built successfully but the rollout never becomes available. Build success proves only that an image exists. Investigate image pull, configuration, scheduling, startup, readiness, and Service endpoint creation as separate stages.

Operational principle: treat every stage transition as an observable contract. The fastest troubleshooting path follows the same graph the platform uses.

TOPIC 2

OpenShift vs Kubernetes: Concepts

Foundation versus enterprise platform experience



Capability	Kubernetes concept	OpenShift experience
Isolation	Namespace	Project adds user-oriented metadata and access workflows
Workload	Deployment / StatefulSet	Same APIs plus integrated topology and platform conventions
Internal access	Service	Service remains the stable in-cluster endpoint
External access	Ingress	Route is an OpenShift API for exposing services
Build	External ecosystem	BuildConfig, S2I and integrated image workflows
Packaging	Helm / manifests	Operators, Helm, templates and software catalog

KEY DISTINCTION OpenShift does not replace Kubernetes. It uses Kubernetes as its foundation and adds integrated APIs, opinionated defaults, supported lifecycle management, and a cohesive user experience.

Example: A standard Kubernetes Deployment and Service can run on OpenShift. A Route, BuildConfig, ImageStream, or SecurityContextConstraint is OpenShift-specific.

KUBERNETES FOUNDATIONS

How Kubernetes Works

Architecture overview based on the CNCF learning model

Kubernetes architecture

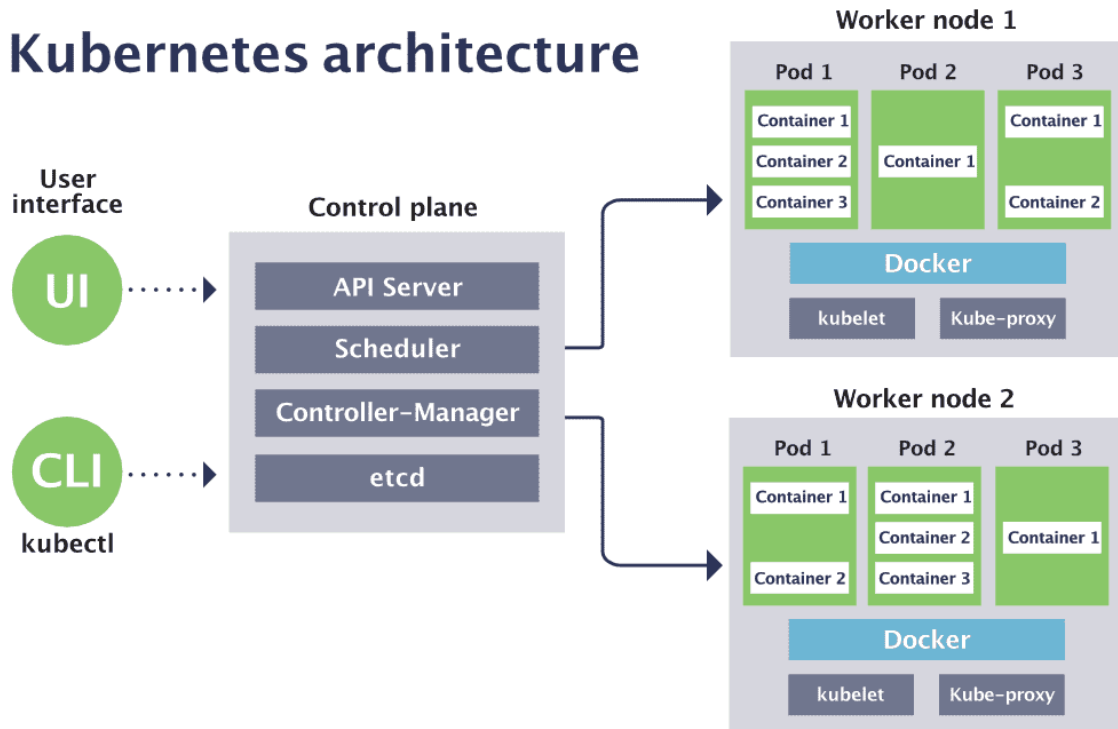


Figure 1. Kubernetes architecture supplied for the workshop. Teaching sequence adapted from CNCF, "How Kubernetes works" (2019).

CURRENT-VERSION CLARIFICATION The diagram labels the node runtime as Docker. Treat this as a historical abstraction. Kubernetes removed its built-in dockershim in v1.24; modern clusters use CRI-compatible runtimes. OpenShift Container Platform uses CRI-O as the container engine.

The architecture separates decision-making from execution. Users and automation submit desired state through the API. Control-plane components validate, store, schedule, and reconcile that intent. Worker nodes run pods and provide node-local runtime and networking services.

KUBERNETES FOUNDATIONS

Terminology from Smallest Unit to Cluster

Pods, Deployments, Services, nodes, control plane, and cluster

Concept	Expert definition	Why it exists
Container	A packaged process with filesystem, configuration and resource isolation	Portable application execution unit
Pod	Smallest schedulable unit; one or more tightly coupled containers sharing network and selected storage	Co-locates processes that must share lifecycle and localhost
Deployment	Declarative controller for replicated stateless pods and rolling updates	Maintains desired replicas and manages revisions
Service	Stable virtual endpoint and DNS identity selecting a changing pod set	Decouples clients from ephemeral pod IPs
Node	Physical or virtual machine running kubelet, runtime and node networking	Provides compute capacity for scheduled pods
Control plane	API-driven decision and reconciliation layer	Maintains cluster state and coordinates nodes
Cluster	Control plane plus worker nodes and supporting infrastructure	Operates workloads as one managed system

POD DESIGN RULE Place containers in the same pod only when they must share network namespace, storage, fate, and scheduling. Independent services belong in separate pods and communicate through Services.

CNCF example generalized: an image-processing pod might contain the request-serving application plus a tightly coupled helper that prepares shared files. Scaling that pod creates another copy of both containers together.

Common misconception: a pod is not a durable server. Controllers replace pods; Services provide the stable access identity. Diagnose the controller and Service relationship rather than trying to preserve a particular pod name or IP.

KUBERNETES ARCHITECTURE

Control Plane Components

How an API request becomes coordinated cluster action

Component	Primary responsibility	What it does not do
kube-apiserver	Exposes the REST API; authenticates, authorizes, admits, validates and persists objects	Does not directly start containers on worker nodes
etcd	Durable distributed key-value store for authoritative cluster state	Is not an application database for workloads
kube-scheduler	Selects a feasible and preferred node for each unscheduled pod	Does not execute the pod or continuously move healthy pods
kube-controller-manager	Runs controllers that reconcile resources such as nodes, replicas and endpoints	Does not replace application-specific business logic
cloud-controller-manager	Integrates cloud nodes, routes and load balancers when applicable	May not exist in the same form on every infrastructure platform

Request lifecycle example: create a Deployment

1. kubectl or oc sends the manifest to kube-apiserver.
2. Authentication, authorization, schema validation, admission and policy checks run.
3. Accepted desired state is stored in etcd.
4. Deployment and ReplicaSet controllers create the required pod objects.
5. The scheduler binds each unscheduled pod to a suitable node.
6. The node kubelet observes the assignment and asks the container runtime to create containers.
7. Status, conditions and events flow back through the API.

IMPORTANT “kubectl apply succeeded” proves that the API accepted the desired state. It does not prove scheduling, image pull, startup, readiness, Service selection, or user reachability.

```
kubectl get deploy,rs,pods -o wide; kubectl get events --sort-by=.metadata.creationTimestamp
```

KUBERNETES ARCHITECTURE

Worker Node Components

Where pods execute and traffic is delivered

Component	Role on the node	Failure evidence
kubelet	Registers the node, watches assigned PodSpecs, manages pod lifecycle, volumes, probes and status	Node NotReady; pod events; kubelet/runtime errors
Container runtime	Pulls images and creates container sandboxes/processes through CRI	ImagePullBackOff, CreateContainerError, runtime service failure
kube-proxy / dataplane	Implements Service forwarding rules where kube-proxy is used	Service IP unreachable despite healthy endpoints
CNI network plugin	Assigns pod networking and programs connectivity/policy	Sandbox creation failure, missing pod IP, denied traffic
CSI storage plugin	Attaches and mounts persistent storage through CSI	Pending PVC, attach/mount events, topology mismatch

Scheduling versus execution: the scheduler chooses a node and records a binding. The kubelet on that node performs execution. A pod can therefore be Scheduled but still fail while pulling an image, mounting storage, creating its network sandbox, starting the process, or passing probes.

NODE FAILURE SCENARIO A node stops reporting heartbeats. The control plane marks it unhealthy. Controllers may create replacement pods elsewhere, but recovery depends on replica count, storage attachability, scheduling constraints, disruption budgets, and available capacity.

```
kubectl get nodes; kubectl describe node/<name>; kubectl get pods --field-selector spec.nodeName=<name>
```

OpenShift mapping: oc provides the Kubernetes resource commands plus OpenShift APIs. On OpenShift nodes, CRI-O is the container engine; cluster networking and node configuration are managed as supported platform components.

TOPIC 2 - RESOURCE MODEL

Desired State and Reconciliation

The mental model that explains most platform behavior

Kubernetes and OpenShift APIs are declarative. A manifest records desired state; controllers continuously compare it with observed state and take actions to reduce the difference. The API response "created" means the object was accepted, not that the application is healthy.

Manifest area	Meaning	Troubleshooting use
apiVersion / kind	Selects the API schema and controller	Confirm the resource exists in the target cluster version
metadata	Identity, namespace, labels, annotations, ownership	Follow labels, ownerReferences and generated names
spec	Desired configuration	Compare intent with actual workload configuration
status	Controller-reported observation	Read conditions, reason, message and timestamps
events	Recent controller and node actions	Build a time-ordered explanation of failure

Ownership chain example: Deployment -> ReplicaSet -> Pod. Deleting a managed pod does not change desired replicas, so the controller creates another. Scaling the Deployment changes desired state and therefore the controller outcome.

DIAGNOSTIC RULE Start with the highest-level object that expresses user intent, then follow ownership downward. Use events and conditions to identify the controller that cannot reconcile.

```
oc get deploy/api -o jsonpath="{.status.conditions}"
oc get pod/ -o jsonpath="{.metadata.ownerReferences}"
oc get events --sort-by=.metadata.creationTimestamp
```

TOPIC 2 - ARCHITECTURE

Architecture Deep Dive

How requests become running workloads

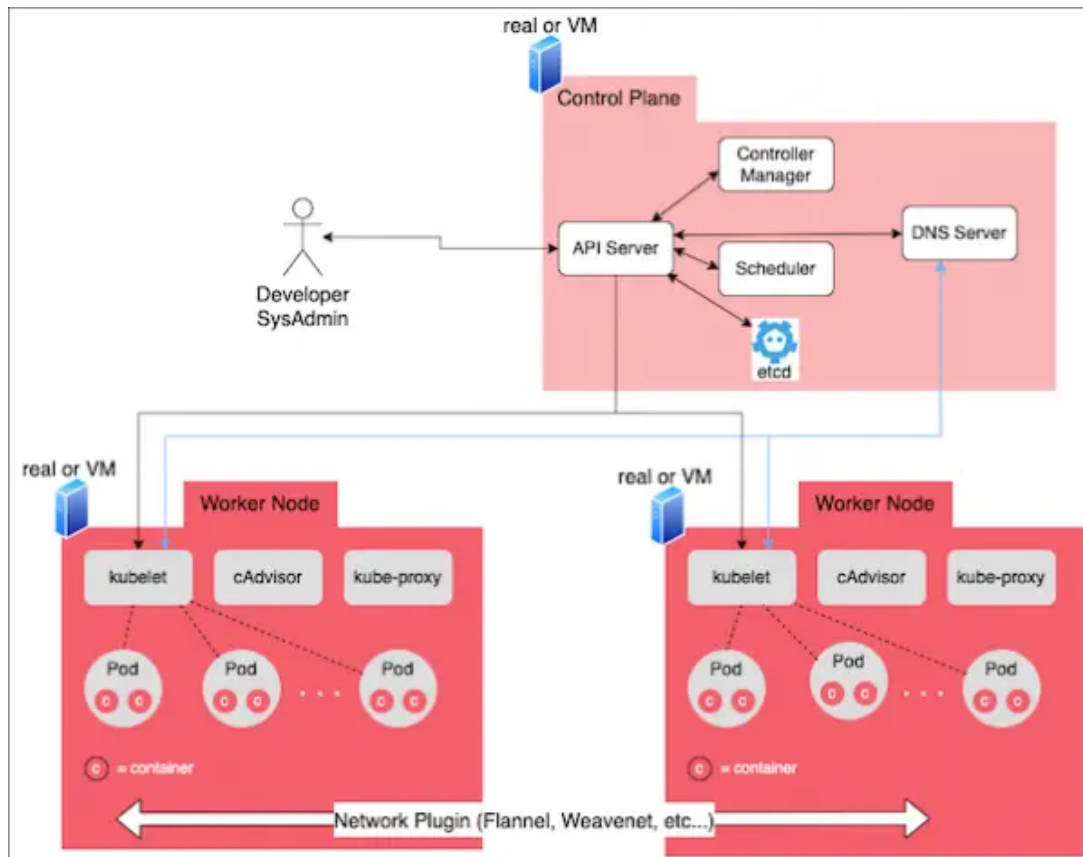


Figure 1. Control plane components and worker-node relationships. Source: Red Hat Developer, Kubernetes 101 for OpenShift developers, Part 1.

Layer	Responsibility	What the participant observes
API and authentication	Validates identity and resource requests	oc and console communicate with the API
etcd	Stores desired and observed state	Objects persist independently of one node
Scheduler/controllers	Select placement and reconcile state	Failed pods are replaced; replicas converge
Workers/runtime	Pull images and run containers	Pods receive IPs and report readiness
Operators	Manage platform and application lifecycle	Status conditions reveal component health

FAILURE SCENARIO A worker node becomes unavailable. The desired replica count remains three. Controllers and the scheduler place replacement pods on healthy workers, subject to capacity and scheduling constraints.

Trace it: user request -> API validation -> desired state in etcd -> controller action -> scheduler placement -> kubelet execution -> status returned to API.

TOPIC 2 - PLATFORM LIFECYCLE

Operators, RHCOS, and Cluster Updates

Why OpenShift is managed as an integrated release

OpenShift uses Operators to encode operational knowledge. Cluster Operators continuously manage core capabilities such as authentication, networking, ingress, image registry, monitoring, and the web console. Red Hat Enterprise Linux CoreOS (RHCOS) provides the managed host operating system, while the Machine Config Operator coordinates node configuration and staged reboots.

Component	Responsibility	Evidence to inspect
Cluster Version Operator	Coordinates supported release payload and update progression	oc get clusterversion; conditions Available/Progressing/Degraded
Cluster Operators	Report capability health and version	oc get clusteroperators
Machine Config Operator	Applies node OS/configuration through machine config pools	oc get mcp; rendered config and degraded nodes
RHCOS	Immutable, cluster-managed host foundation	Node version, kubelet/runtime status
Operator Lifecycle Manager	Installs and upgrades optional Operators	Subscriptions, InstallPlans, CSV conditions

OPERATIONS SCENARIO An update pauses because the worker machine config pool is degraded. Do not retry blindly. Identify the degraded node and MCO reason, correct configuration drift or node health, and confirm the pool returns Updated before continuing.

`oc get clusterversion; oc get co; oc get mcp`

Workshop boundary: developers usually consume these capabilities rather than administer them, but understanding their status prevents misdiagnosing a platform-wide issue as an application defect.

TOPIC 3

Projects, Services, and Routes

The resource relationships behind a reachable application

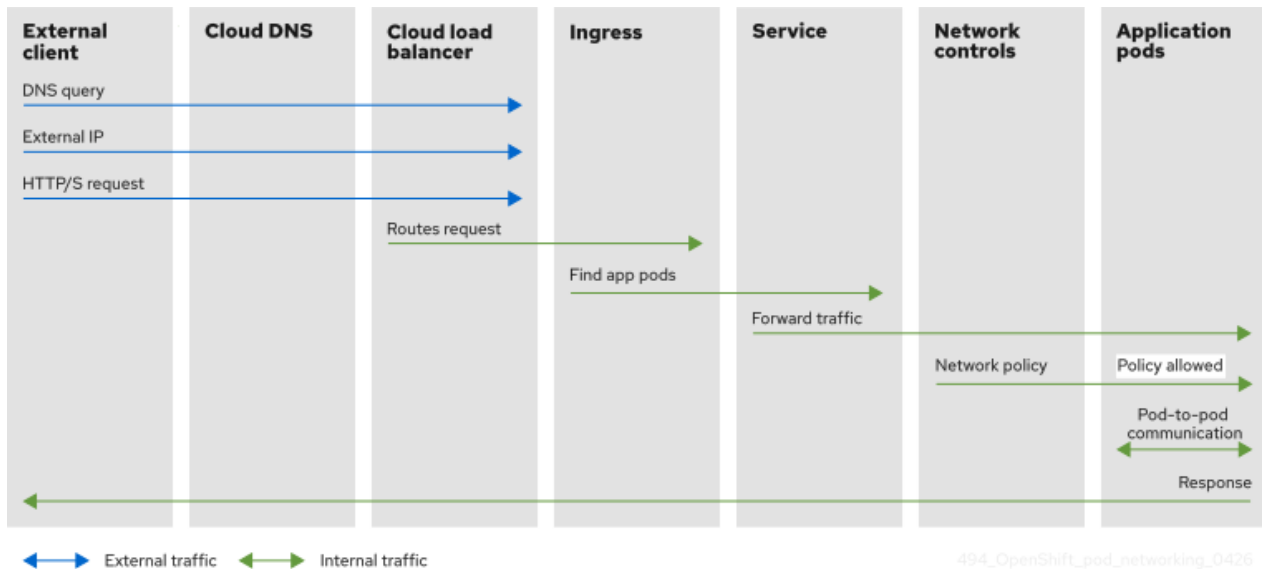


Figure 2. External and internal traffic through DNS, load balancer, Ingress, Service, network controls, and application pods. Source: Red Hat OpenShift Container Platform 4.20 Networking overview.

Object	Purpose	Common diagnostic
Project	Tenant boundary for resources, access, quotas, and policies	oc project; oc get rolebindings
Pod	Running instance of one or more containers	oc get pods; oc describe pod
Service	Stable endpoint selecting pods by labels	oc get endpoints; compare selectors
Route	External hostname forwarding to a Service	oc get route; inspect TLS and target port

REAL-TIME SCENARIO The route opens but returns 503. Work backward: Route target Service -> Service port/targetPort -> Endpoints -> pod labels -> readiness -> application listener.

oc new-project workshop-demo | oc get svc,route,pods | oc describe route sample

TOPIC 3 - TENANCY

Projects as Security and Operating Boundaries

Names, access, quotas, defaults, and lifecycle

An OpenShift project is backed by a Kubernetes namespace and adds a user-oriented project API and workflows. It is a boundary for namespaced resources, RBAC, quotas, limit ranges, network policy, service accounts, and many operational reports. It is not a hard multi-tenant boundary by itself; isolation depends on the policies applied to it.

Control	What it governs	Failure symptom
RoleBinding	Who can perform verbs on namespaced resources	Forbidden response from API
ResourceQuota	Aggregate project consumption or object counts	Exceeded quota during create/update
LimitRange	Default/min/max container requests and limits	Rejected pod or unexpected defaults
NetworkPolicy	Allowed pod ingress/egress relationships	Timeout despite healthy pods and Services
ServiceAccount	Workload identity used for API and image access	Image pull or API authorization failure

DESIGN DECISION Separate environments when access, lifecycle, blast radius, quota, or promotion policy differs. Do not create projects solely to group every microservice; use labels and application metadata for grouping.

`oc describe quota; oc describe limitrange; oc get rolebinding; oc auth can-i --list`

Example: a production project may deny developer mutation while permitting a GitOps service account to apply approved manifests. The same team can retain edit access in development.

TOPIC 3 - DIAGNOSTICS

Networking Troubleshooting Playbook

Follow the packet path and prove each handoff

Layer	Question	Evidence / command
DNS and route admission	Does the hostname resolve and is the Route admitted?	oc get route -o wide; route status ingress conditions
Ingress to Service	Does the Route target the intended Service and port?	oc describe route; oc get svc -o yaml
Service selection	Do selectors match ready pods?	oc get pods --show-labels; oc get endpointslices
Application listener	Is the process listening on targetPort and correct interface?	oc rsh; curl localhost;; pod logs
Policy and TLS	Is traffic denied or TLS mode inconsistent?	NetworkPolicy, router logs, certificate/termination settings

INCIDENT SCENARIO A Route returns 503 only after a rollout. The new pods are Running but not Ready. Endpoints contain only the old pod until it terminates; after that the Service has none. Root cause: readiness probes use the wrong path. Correct the probe, not the Route.

Evidence sequence

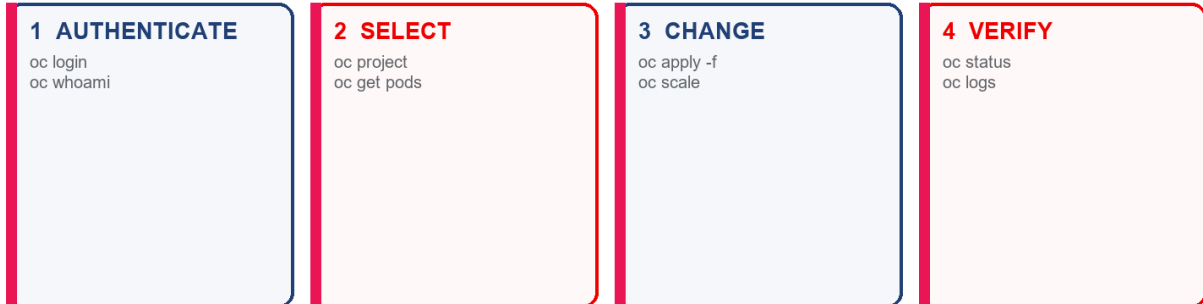
```
oc get route,svc,endpointslice,pods -l app=orders -o wide
oc describe route/orders; oc describe svc/orders
oc describe pod/; oc logs --previous
```

Expert habit: avoid changing multiple layers simultaneously. Record the failing handoff, make one targeted correction, then retest from the same client path.

TOPIC 4 - OC CLI

Working with the oc CLI

A repeatable observe-change-verify workflow



Intent	Command example	What to check
Context	oc whoami; oc project	Correct user and project
Inventory	oc get all; oc get events	Workload state and warnings
Inspect	oc describe pod/	Conditions, probes, mounts and events
Logs	oc logs deploy/ --tail=100	Startup and request failures
Apply	oc apply -f app.yaml	Acceptance and configured resource
Rollout	oc rollout status deploy/	Progress and replica readiness

SAFE OPERATING HABIT Before every change, confirm identity and project. After every change, verify status and events. Resolve the exact resource and namespace before destructive operations.

oc get pods -> oc describe pod api-... -> oc logs deploy/api -> oc get events --sort-by=.lastTimestamp

TOPIC 4 - CLI PRACTICE

oc Patterns for Daily Engineering

Selectors, output formats, patches, and safe changes

Pattern	Example	Why it matters
Label selection	<code>oc get pods -l app=orders,env=dev</code>	Targets a logical set instead of generated names
Structured output	<code>oc get deploy/orders -o jsonpath="{.status.readyReplicas}"</code>	Supports scripts without scraping tables
Diff before apply	<code>oc diff -f manifests/</code>	Shows server-side change before mutation
Server-side apply	<code>oc apply --server-side -f manifests/</code>	Tracks field ownership and supports declarative collaboration
Controlled edit	<code>oc patch deploy/orders --type=merge -p ...</code>	Small auditable change without replacing the object
Wait for condition	<code>oc wait --for=condition=Available deploy/orders --timeout=120s</code>	Turns reconciliation into an explicit gate

SAFETY NOTE Imperative commands are excellent for exploration and incidents. Durable environment changes should be captured in version-controlled manifests or GitOps so desired state can be reviewed and reproduced.

```
oc explain deployment.spec.template.spec.containers --recursive
oc get pods -l app=orders -o
custom-columns=NAME:.metadata.name,READY:.status.containerStatuses[0].ready,NODE:.spec.nodeName
oc set resources deploy/orders --requests=cpu=100m,memory=128Mi --limits=cpu=500m,memory=512Mi
```

Lab task: scale an application, wait for availability, select the new pods by label, and produce a compact table containing pod name, readiness, restart count, and node.

TOPIC 4 - WEB CONSOLE

Web Console Navigation

Use topology and resource pages to tell the application story

Console area	Use it to	Facilitator demonstration
Project selector	Set the working scope	Switch projects and show inventory changes
Topology	See workloads, routes and rollout status	Open a component and follow resource links
Workloads	Inspect Deployments, pods and YAML	Scale replicas and watch rollout progress
Networking	Inspect Services and Routes	Copy the route URL and validate endpoint
Observe	Read metrics, logs, events, and alerts	Correlate a failed probe with pod events

GUIDED SCENARIO A retail catalog shows 'Application is not available.' In Topology, the workload has a red badge. Open Resources -> pod -> Events, identify an image pull error, correct the image reference, and watch the rollout become ready.

What you click	Equivalent CLI starting point
Project > Workloads	<code>oc get deploy,pods</code>
Pod > Logs	<code>oc logs pod/</code>
Route details	<code>oc describe route/</code>
Deployment > YAML	<code>oc get deploy/ -o yaml</code>

Version note: Starting with OpenShift 4.19, console navigation is unified and the Developer perspective is no longer enabled by default. Administrators can enable it when required.

Facilitator tip: Ask: Which object owns this pod? Which Service selects it? Which Route exposes that Service?

TOPIC 4 - GUIDED LAB

Web Console Guided Investigation

Translate visual status into resource evidence

Scenario: the inventory service deployment shows a degraded topology badge after a configuration change. Your task is to identify the first failed contract and recommend a reversible correction.

Step	Console action	Evidence to capture
1. Set context	Choose the assigned project and verify the project name	Project and signed-in user
2. Open topology	Select inventory and open Resources	Workload kind, desired/ready replicas, active revision
3. Inspect pods	Open the newest pod and review Details/Events	Condition reason, scheduling or probe message
4. Read logs	Open current and previous container logs	First application-level error and timestamp
5. Compare YAML	Review env, ConfigMap/Secret refs, ports and probes	Changed field linked to the failure
6. Correct and verify	Apply the approved edit and watch rollout	New revision available; old replica terminated safely

FACILITATOR CHECKPOINT Do not accept "the pod is red" as a diagnosis. Require the object, condition, reason, message, and supporting log/event evidence.

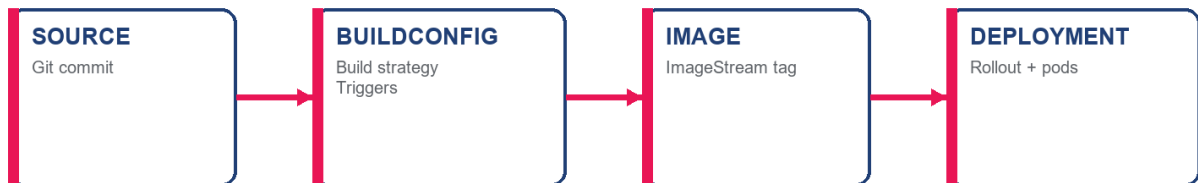
CLI cross-check: `oc get deploy,pods; oc describe pod/<name>; oc logs pod/<name> --previous; oc rollout status deploy/inventory`

Debrief: Which information was faster to discover visually? Which evidence was more precise in CLI output? How would you record the correction for repeatability?

TOPIC 5

BuildConfig and DeploymentConfig

A source-to-image-to-rollout learning model



BuildConfig defines how source becomes an image and what triggers a build. DeploymentConfig is an OpenShift-specific historical deployment API. Modern environments commonly favor Kubernetes Deployments for new applications; use DeploymentConfig where existing standards or workloads require it.

Stage	Example action	Verification
Create build	oc new-build ~ --name=orders	oc get bc; oc logs -f bc/orders
Start build	oc start-build orders --follow	Build completes and ImageStream updates
Create app	oc new-app orders:latest	Workload, Service and pods appear
Expose	oc expose service/orders	Route hostname is admitted
Update	Commit source or start a new build	Configured trigger starts rollout

REAL-TIME SCENARIO A Java team commits a security fix. A Git trigger starts an S2I build, the image updates its ImageStream tag, and an image-change trigger starts a controlled rollout. If readiness fails, traffic remains with ready replicas.

TOPIC 5 - BUILD INTERNALS

BuildConfig, ImageStreams, and S2I

What the build controller creates and why image identity matters

A BuildConfig describes build inputs, strategy, output, and triggers. Source-to-Image (S2I) combines application source with a builder image to produce a runnable image. ImageStreams provide stable OpenShift references to image tags and record image metadata/digests; they can trigger builds or deployments when tags change.

BuildConfig area	Question to ask	Common failure
source	Which Git URI, ref, contextDir, and credentials are used?	Wrong branch, missing secret, inaccessible repository
strategy	S2I, Docker, or custom? Which builder image?	Builder tag missing or incompatible source
output	Which ImageStreamTag or external registry receives the image?	Push authentication or quota failure
triggers	GitHub, Generic, ConfigChange, ImageChange, manual?	Unexpected or absent build
resources / volumes	Does the build have adequate compute and trusted inputs?	OOMKilled, timeout, missing certificate

Annotated skeleton

```

apiVersion: build.openshift.io/v1
kind: BuildConfig
metadata: {name: orders}
spec:
  source: {type: Git, git: {uri: https://git.example/orders.git}}
  strategy: {type: Source, sourceStrategy: {from: {kind: ImageStreamTag, name: java:17}}}
  output: {to: {kind: ImageStreamTag, name: orders:latest}}
```

EXPERT NOTE latest is convenient for training but ambiguous for promotion. Production delivery should retain immutable image digests and provenance so the exact artifact can be verified and rolled back.

TOPIC 5 - ROLLOUT MECHANICS

DeploymentConfig vs Deployment

Legacy compatibility, current direction, and rollout evidence

Dimension	DeploymentConfig	Kubernetes Deployment
API	apps.openshift.io/v1	apps/v1
Replica controller	ReplicationController	ReplicaSet
Triggers	Built-in ConfigChange and ImageChange	Typically external automation or image update tooling
Strategies	Rolling, Recreate, Custom	RollingUpdate, Recreate
Hooks	Lifecycle hooks supported	Use init containers, Jobs, pipelines or operators
Current guidance	Understand for existing applications	Preferred general-purpose API for new stateless workloads

Rolling rollout mechanics: maxSurge controls temporary extra replicas; maxUnavailable limits capacity removed during update. Readiness gates traffic and rollout progress. progressDeadlineSeconds marks stalled rollouts, but does not repair the cause.

FAILURE SCENARIO A rollout stalls at one unavailable replica. Events show readiness probe failures; logs show the application requires a database migration. Decide whether to run a pre-deployment Job, make startup backward-compatible, or use Recreate when simultaneous versions are unsafe.

```
oc rollout history deploy/orders; oc rollout status deploy/orders
```

```
oc rollout undo deploy/orders --to-revision=<n>
```

Decision test: Can old and new versions run concurrently against the same dependencies and data model? If not, a rolling strategy may be unsafe even when it is technically available.

TOPIC 5 - END-TO-END LAB

Lab: Build, Deploy, Expose, Update

A complete evidence-driven application workflow

Phase	Participant action	Acceptance evidence
Prepare	Create/select project; confirm quota and identity	Correct user/project; sufficient quota
Build	Create BuildConfig from approved Git source; follow logs	Build Complete; output ImageStream digest recorded
Deploy	Create Deployment referencing the image; set requests and probes	Available condition true; ready replicas match desired
Expose	Create Service and edge-terminated Route	Route admitted; HTTPS response returns expected version
Update	Change source; start/trigger build; watch rollout	New digest deployed; rollout completes without endpoint gap
Diagnose	Introduce a bad readiness path and repair it	Evidence identifies probe; corrected revision becomes available

```
oc new-project app-lab; oc new-build java:17~https://git.example/orders --name=orders
oc logs -f bc/orders; oc new-app orders:latest; oc expose svc/orders
oc set probe deploy/orders --readiness --get-url=http://:8080/health/ready
oc rollout status deploy/orders; oc get route orders -o jsonpath="{.spec.host}"
```

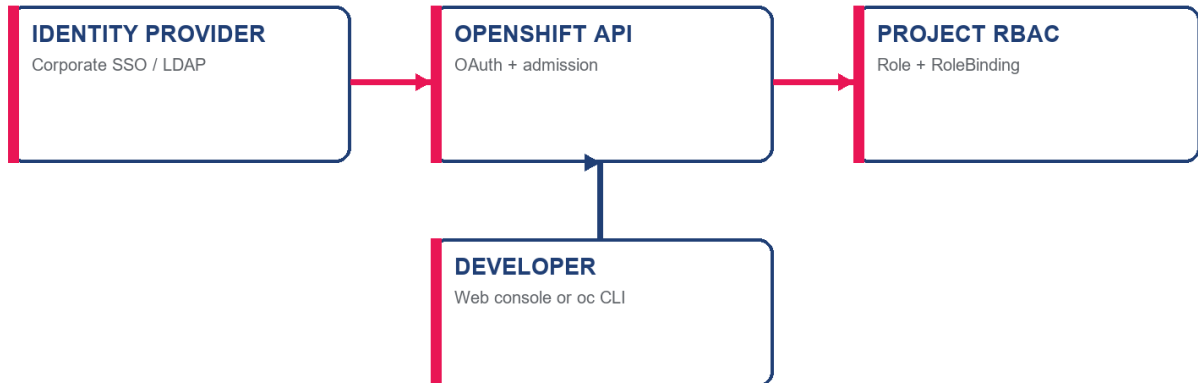
LAB DELIVERABLE Submit the build name, image digest, Deployment revision, ready pod names, Service endpoints, Route host, and a short explanation of one failure using conditions/events/logs.

Stretch: add a ConfigMap value, trigger a rollout through a pod-template annotation, and explain why changing a ConfigMap alone might not restart existing pods.

TOPIC 6

Platform Architecture and Developer Access

Identity, RBAC, project boundaries, and shared responsibility



Actor	Typical responsibility	Least-privilege example
Cluster administrator	Lifecycle, Operators, policy and identity providers	cluster-admin only for designated operators
Project administrator	Project access, quotas and coordination	admin role within one project
Developer	Build, deploy, inspect and troubleshoot	edit role in team projects
Viewer / auditor	Read state, logs or selected reports	view role without mutation rights
Service account	Workload-to-API identity	Dedicated narrowly scoped RoleBinding

ACCESS SCENARIO A developer can view pods but receives 'forbidden' when creating a Route. Diagnose with `oc auth can-i` create routes and inspect RoleBindings. Add only the required project role; do not solve a project permission issue with cluster-admin.

Checklist: map corporate identity; prefer group grants; separate dev/test/prod; enforce quotas and limits; review service-account permissions separately.

TOPIC 6 - AUTHORIZATION

RBAC and Workload Identity

Subjects, roles, bindings, and permission diagnosis

RBAC object	Scope and purpose	Example
Role	Namespaced permission rules	Allow get/list/watch on pods in one project
ClusterRole	Reusable cluster-scoped rule set	Built-in view, edit, admin; or custom cross-namespace rule
RoleBinding	Grants Role/ClusterRole within one namespace	Grant edit to a developer group in dev
ClusterRoleBinding	Grants ClusterRole across the cluster	Reserved for platform-wide operational identities
ServiceAccount	Namespaced workload identity	Pipeline deployer or application API client

Permission evaluation: RBAC is additive; there is no explicit deny rule. Effective authorization may also be constrained by admission, SCCs, quota, or policy. A successful can-i result does not guarantee the object will pass all admission checks.

```
oc auth can-i create routes -n team-dev
```

```
oc auth can-i --as=system:serviceaccount:team-dev:pipeline apply deployments
```

```
oc adm policy who-can delete secrets -n team-dev
```

SECURITY SCENARIO A pipeline can deploy workloads but cannot read arbitrary Secrets. Create a dedicated Role containing only required verbs/resources and bind it to the pipeline service account. Avoid reusing the default service account or granting edit when narrower permissions work.

Review question: Is the identity human or workload? What namespace should the grant affect? Which verbs and resources are essential? Can the grant be assigned to a group rather than individuals?

TOPIC 6 - RUNTIME SECURITY

Security Context, SCCs, Secrets, and Supply Chain

How platform guardrails affect application design

Control	Protects against	Developer responsibility
Security Context Constraints	Unsafe privilege, host access, identity and capability settings	Build images that run with arbitrary non-root UID and minimal capabilities
Secrets and ConfigMaps	Hard-coded configuration and credentials	Mount only required values; rotate and avoid logging sensitive data
NetworkPolicy	Unrestricted lateral movement	Declare allowed ingress/egress dependencies
Image policy / signatures	Unknown or untrusted artifacts	Use approved registries, immutable digests, and provenance
Requests, limits, quota	Noisy-neighbor and capacity exhaustion	Profile workloads and set realistic resource envelopes

COMMON FAILURE An image works locally but fails on OpenShift with permission denied. It writes to a root-owned directory and assumes UID 1000. Fix the image: use group-writable paths, avoid fixed UID assumptions, and write only to declared writable locations.

Container image checklist

- No embedded credentials or private keys
- Runs as non-root/arbitrary UID
- Uses read-only root filesystem where practical
- Has explicit health endpoints
- Emits logs to stdout/stderr
- Pins dependencies and records source/SBOM
- Handles SIGTERM and graceful shutdown

Important: SecurityContextConstraints are OpenShift-specific admission controls. Pod Security Admission and other policies may coexist; always inspect the cluster's actual policy and error message.

OPERATIONS - OBSERVABILITY

Observability and Incident Reasoning

Conditions, events, logs, metrics, and traces answer different questions

Signal	Best for	Limitations
Conditions/status	Current controller interpretation and progression	May summarize rather than expose root cause
Events	Recent scheduling, pulling, mounting and probe actions	Time-limited; not a durable audit trail
Container logs	Application and runtime messages	Only useful if structured and correlated
Metrics	Trends, saturation, error rate and alerting	Requires context and meaningful thresholds
Distributed traces	Cross-service latency and dependency path	Requires instrumentation and sampling strategy

INCIDENT SCENARIO Checkout latency rises after a release. Pods are Ready and CPU is normal. Metrics show database connection saturation; traces show waiting in the payment dependency. The platform is healthy, but the application dependency contract is not.

Evidence-driven incident loop

1. State the user-visible symptom and time window.
2. Confirm scope: one pod, one project, one node, or cluster-wide.
3. Compare desired and observed state.
4. Follow ownership and traffic/dependency paths.
5. Correlate conditions, events, logs and metrics by timestamp.
6. Make one reversible change and verify the original symptom.
7. Record cause, contributing controls, and prevention action.

```
oc get events --sort-by=.lastTimestamp; oc adm top pods; oc logs deploy/checkout --since=15m
```

REFLECTION & REVIEW

Connect the objects, commands, and responsibilities

Facilitated recap and knowledge check

Core question	Expected explanation
What does OpenShift add?	Integrated capabilities and supported, opinionated workflows on Kubernetes.
How does traffic reach a pod?	Client -> Route -> Service -> ready endpoints selected by labels.
What is a project?	A user-oriented namespace boundary for resources, access, quotas, and policy.
How do you troubleshoot?	Confirm context, inspect status/events, follow relationships, verify changes.
How does source become a rollout?	BuildConfig creates an image; a configured trigger can start deployment.

CAPSTONE SCENARIO The orders API route returns 503 after a new build. Produce a six-command investigation connecting Build -> Image -> Workload -> Pod readiness -> Service endpoints -> Route.

`oc project | oc get route,svc,endpoints | oc get pods | oc describe pod/ | oc logs deploy/orders | oc rollout status deploy/orders`

Reflection prompts: One concept I can explain; one command I will use first; one guardrail that improves speed; one question I still need to test.

Further reading: CNCF How Kubernetes Works; Kubernetes Concepts and dockershim-removal guidance; Red Hat OpenShift 4.20 Architecture, Nodes and CLI tools; OpenShift 4.21 Web Console and Projects.

Sources: <https://www.cncf.io/blog/2019/08/19/how-kubernetes-works/> | <https://kubernetes.io/docs/concepts/> | <https://kubernetes.io/blog/2022/02/17/dockershim-faq/> | https://docs.redhat.com/en/documentation/openshift_container_platform/4.20/html/nodes/working-with-containers | https://docs.redhat.com/en/documentation/openshift_container_platform/4.20/html/architecture/architecture-overview | https://docs.redhat.com/en/documentation/openshift_container_platform/4.21/html-single/web_console/index